CHAPTER 06 DATA ENGINEERING / SEMI-STRUCTURED

JSON and Semi-Structured Data

Turning changing API payloads into dependable analytical facts

The dashboard did not break. Its meaning did.
At 08:00, the energy dashboard reports the correct daily total. At 09:00, a supplier adds a nested `unit` field, changes `observations` to `measurements`, and represents a missing reading as JSON `null`. The ingestion job still says *success*. A naive flattening query silently returns zero observations for the new payload. The next chart is empty, not because production stopped, but because the data contract changed.

In Chapter 5, we preserved exact file provenance before accepting source records. This chapter extends the same discipline to nested data. A JSON document is useful because it preserves the source's hierarchy; a data warehouse is useful because it gives that hierarchy consistent analytical meaning. Connecting the two requires more than a cast and a FLATTEN.

What you will learn

- decide when to retain flexible JSON in VARIANT and when to normalize it;
- distinguish parse errors, missing paths, JSON `null`, SQL NULL and wrong types;
- traverse objects and arrays and reason about the number of rows produced;
- handle publisher schema revisions without hiding invalid observations;
- turn mixed-version energy API messages into typed, tested silver-layer records.

Prerequisites. Chapters 4–5, basic SELECT/JOIN, and familiarity with a JSON object. The companion includes synthetic fixtures and an offline Python validator, requiring no cloud account. Optional Snowflake SQL requires an approved limited role and a small, auto-suspending warehouse. No Snowflake execution or performance benchmark is claimed for this manuscript.

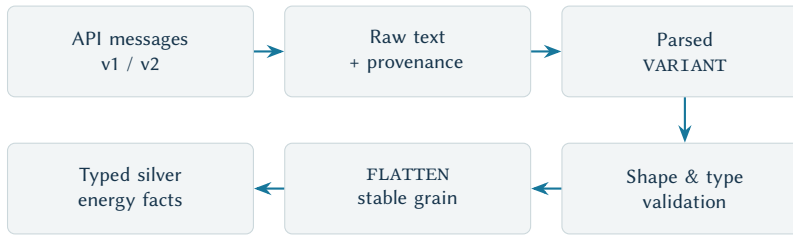


Figure 6.1. Preserve the original message while promoting only validated readings.

6.1 Semi-structured data and the VARIANT type

6.1.1 Semi-structured payloads and VARIANT

A relational table gives each column a declared type and each row an expected shape. API documents often do not. A payload may combine an envelope, a provider identifier, a timestamp, an optional quality flag and a variable number of measurements. Schema variation is not necessarily a data-quality failure: the publisher may be introducing an optional field legitimately. The engineering challenge is to preserve useful flexibility without abandoning predictable outputs.

Snowflake's VARIANT type stores a single semi-structured value, including a JSON-derived object, array, scalar or null. It is not the same as putting opaque JSON *text* into a VARCHAR. Parsing interprets the hierarchy, so queries can navigate paths and work with nested values. Yet the original text may remain important for legal provenance, replay, cryptographic comparison or investigating a parse failure. A robust bronze boundary can preserve both the exact text and a parsed representation, with separate source identifiers.

Listing 1: Inspect the type after parsing a synthetic API message.

```
SELECT TYPEOF(PARSE_JSON(
  '{"provider":"synthetic-fi","observations":[{"mwh":12.5}]}'
)) AS document_type;
-- Expected type: OBJECT (illustrative, not cloud-tested).
```

The most important distinction is **schema flexibility versus semantic ambiguity**. A publisher can change the arrangement of fields without altering the meaning of *energy measured during the UTC interval.* But if a field called `value` changes from MWh to kWh, a successful PARSE_JSON does nothing to prevent a thousandfold reporting error. Structure and meaning require separate contracts.

A two-layer contract

- *Envelope contract:* source ID, retrieval time, schema version, valid JSON, payload hash, publisher terms and message identity.
- *Observation contract:* metric identity, UTC observation time, unit, numeric value, acceptable domain and deduplication key.

The first contract helps you replay a message accurately; the second tells you which rows should count in a report. Do not confuse storing a document with accepting every measurement it contains. Snowflake's documented guidance permits keeping the flexible structure in VARIANT while promoting high-use attributes into typed columns when workloads are understood.

Project reference: P058: Document AI introductory reference. *Not independently executed; rights review pending.*

6.1.2 Parsing types and nested values

JSON's small type system creates surprisingly large modelling decisions. Its values may be objects, arrays, strings, numbers, booleans or `null`; it does not define a native timestamp, binary data type or the business meaning of a measurement. The string `"12.50"` and numeric value `12.50` may look identical in a dashboard, but a typed transformation should treat them differently until an explicit contract authorizes conversion.

Incoming form	Example and engineering response
Numeric JSON value	`12.50`; check range and units before using
Numeric string	`"12.50"`; parse deliberately and record conversion
Timestamp string	`"2026-01-15T10:00:00Z"`; normalize to UTC
Object or array	Nested provider information; validate expected structure
JSON null	Explicitly supplied null; apply missing-data policy
Missing key	No supplied value; distinguish from explicit null

`TYPEOF` and type predicates help diagnose unexpected values. Consider a measurement property that was a number yesterday but becomes an object with `amount` and `unit` keys today. A blind cast may raise an error, or a try-cast may yield SQL `NULL`. Neither outcome tells the analyst *why* it happened. Your schema-drift report should record the encountered path and type, not simply discard the row.

Listing 2: Inspect the nested value before applying a numeric contract.

```
WITH src AS (
  SELECT PARSE_JSON('{"value":{"amount":12.5}}') AS v
)
SELECT TYPEOF(v:value) AS value_kind,
       TYPEOF(v:value:amount) AS amount_kind,
       v:value:amount::VARCHAR AS raw_amount
FROM src;
```

The shape `OBJECT` is expected for the outer `value`; the inner `amount` is a number. Cast only after the path and shape have been checked. Native JSON numeric literals may preserve fixed-point values, but the parsing engine can use floating representation for scientific notation or values outside fixed-point bounds. Never infer measurement precision from a screenshot of pretty-printed JSON.

Practical judgment. The bronze schema should not be redesigned every time a nonessential source field appears. The silver schema *should* change through an explicit review whenever a published business metric changes meaning. Keep this separation throughout the book.

6.1.3 Choosing schema-on-read versus early normalization

Schema-on-read means preserving a relatively flexible representation and interpreting selected paths when queried. *Early normalization* means parsing, validating and promoting the important attributes into relational columns soon after ingestion. Neither approach is a universal winner. The design should follow the consumer's contract and the likely failure modes.

Use a flexible bronze object when a new source is exploratory, nested fields change frequently, or replay of previously unseen attributes is valuable. Use typed silver columns for stable KPIs, joins, constraints, timestamps and large repeated workloads. In the Finland Energy Analytics platform, bronze retains the provider's JSON, while silver represents one row per `provider + technology + UTC interval`. An analytical aggregate must never depend on whether the provider called an array `observations` or `measurements`.

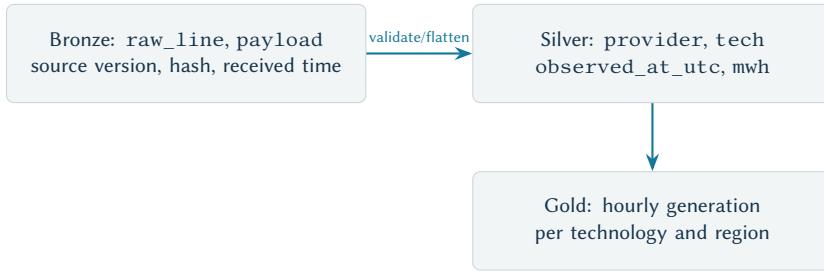


Figure 6.2. Promote a well-defined business grain, not an accidental JSON shape.

A less obvious issue is *reprocessing*. A schema correction might require recreating a silver table from last week's original messages. If bronze keeps only a destructive transformation and discards the payload, the repair may be impossible without retrieving the source again. Preserving exact source text and publishing a migration plan makes that recovery feasible.

Decision rule.
Choose `VARIANT` as a flexible boundary, not an excuse to delay contracts forever. Materialize typed, frequently queried fields after measurement, and retain original content long enough for your documented audit and replay obligations. The appropriate retention period depends on rights, cost and organizational policy.

Check. Identify a new optional provider tag that should remain flexible, and a unit-of-measure change that must block publication until approved. Explain why they deserve different treatment.

6.2 OBJECT and ARRAY structures

6.2.1 Constructing OBJECT values

An OBJECT represents named attributes, not a positional record. It is useful for packaging a set of related fields while preserving natural source hierarchy. Snowflake provides OBJECT_CONSTRUCT; its treatment of SQL NULL values is significant: key/value pairs with a SQL-null value are omitted, whereas OBJECT_CONSTRUCT_KEEP_NULL retains them as JSON nulls. This is more than a formatting choice when a downstream consumer distinguishes *not provided* from *explicitly cleared*.

Listing 3: Construct a compact provider object with explicit null behavior.

```
SELECT OBJECT_CONSTRUCT(
         'provider', 'synthetic-fi',
         'revision', 2,
         'note', NULL
       ) AS omitted_note,
       OBJECT_CONSTRUCT_KEEP_NULL(
         'provider', 'synthetic-fi',
         'revision', 2,
         'note', NULL
       ) AS retained_note;
```

An object has named keys, but do not assume any stable order of keys when printed or converted back to JSON text. Source hashes should be calculated over the original source bytes, not by assuming TO_JSON(PARSE_JSON(x)) reproduces those bytes exactly. Whitespace, key order and normalization can change.

6.2.2 Constructing ARRAY values

An ARRAY represents a sequence. In our payload it contains readings for one observation interval. Arrays are zero-indexed in JSON-style path operations, and position can matter when order encodes source priority or telemetry sequencing. Use ARRAY_CONSTRUCT to build illustrative sequences; use ARRAY_SIZE to inspect expected cardinality, and GET to retrieve elements dynamically.

Listing 4: Object versus array: names compared with positions.

```
SELECT ARRAY_CONSTRUCT('wind','solar','hydro') AS techs,
       GET(ARRAY_CONSTRUCT('wind','solar','hydro'),1)
         AS second_technology;
-- Index 1 refers to the second array element.
```

For ingestion, an empty array is not equivalent to a missing array or a malformed array. An empty array may be a legitimate “no readings” batch; a missing field may signal schema drift; an object where an array was promised is a contract violation. Capture each case separately.

Project reference: P059: Unstructured-data extraction reference. *Not independently executed; rights review pending.*

6.2.3 Safe traversal and array/object type inspection

Paths are not a substitute for shape checks. Suppose a supplier sends a `meter` object for one row and an array of `meter` objects for another. Both parse as valid JSON, but the query `payload:meter:id` only expresses the first structure. A robust mapper either supports both variants by version or rejects the unexpected version explicitly.

For diagnostic exploration, ask three questions in order: Does the path exist? What type is its value? Is the value allowed by the business contract? A path can exist and contain JSON null; it can be a number when a string is expected; or it can be an array of objects instead of one object. These distinctions determine whether to default, quarantine or migrate.

Listing 5: Non-destructive shape inspection of an array field.

```
WITH src AS (
 SELECT PARSE_JSON('{"observations":[{"tech":"wind"}]}') v
)
SELECT TYPEOF(v) AS document_kind,
       TYPEOF(v:observations) AS readings_kind,
       ARRAY_SIZE(v:observations) AS readings_count
FROM src;
```

Array cardinality is a data contract. If a publisher guarantees one reading per technology, duplicated technologies in an array can double totals unless the consumer enforces a uniqueness rule. Conversely, a missing technology might mean the source did not report it; automatically writing zero would make absence look like measured zero. For this book, each accepted observation is keyed by provider, technology and UTC interval. An array position is retained as provenance, not used as a stable business key.

Which default is safe?
A missing `comment` can normally remain `NULL`. A missing `unit` is unsafe when MWh and kWh may coexist. An empty `observations` array should trigger a source completeness check; a JSON null `mwh` must never be summed as though it were an actual zero.

An automated test fixture should cover valid object, empty array, missing array, wrong field type, duplicate reading key and explicit null. These cases cost little to construct and detect a surprising range of production failures.

6.3 Parsing, inspecting, and validating JSON

6.3.1 PARSE_JSON and TRY_PARSE_JSON

PARSE_JSON interprets JSON text and returns a VARIANT; a malformed nonempty input raises an error. TRY_PARSE_JSON returns SQL NULL when parsing fails. That behavior is valuable for quarantine, but it is not a complete validity test: null input, empty or whitespace-only input, and invalid JSON require explicit handling. In Snowflake, the text `'null'` parses to a JSON null VARIANT, which differs from an SQL-null parse result.

Listing 6: Observe four distinct parser outcomes.

```
SELECT TRY_PARSE_JSON('{"a":1}') AS valid_object,
       TRY_PARSE_JSON('{bad json}') AS invalid_text,
       PARSE_JSON('null') AS json_null,
       PARSE_JSON(NULL) AS sql_null;
```

6.3.2 Invalid payload detection

Do not simply discard rows where TRY_PARSE_JSON returns null. Preserve the raw input, manifest, position, error class and failed contract version. A failure may be due to invalid syntax, an empty body, unsupported shape, duplicate object key or a valid document with invalid measurements. Snowflake's strict JSON parser rejects duplicate object keys by default; permissive duplicate-key parsing is a deliberate trade-off because the last value wins and information is lost.

6.3.3 Test fixtures for valid and malformed JSON

The companion file `data/sample_api_payload.jsonl` includes two valid envelope revisions, one syntactically valid but semantically unacceptable envelope and one malformed line. That arrangement tests both parsing and business acceptance. The offline verifier reports the original line number without sending payloads to an external service.

Testing invariant.
Every physical input line must have exactly one recorded outcome: *accepted envelope*, *quarantined syntax*, or *quarantined contract*. The sum of these categories must equal the physical input line count. Never confuse "the parser did not throw" with "the data are fit for analytics."

Project reference: P060: Community JSON pipeline. *Not independently executed; rights review pending.*

Project reference: P061: Streams-and-tasks JSON pipeline. *Not independently executed; rights review pending.*

6.4 Querying nested keys and paths

6.4.1 Path notation for nested attributes

A path such as `payload:source:provider` is shorthand for selecting a nested attribute. Array positions use brackets, for example `payload:observations[0]:tech`. A valid path can return a VARIANT, not yet the typed string or number your model needs. Quoted key syntax and GET_PATH are helpful when key names contain punctuation or are dynamic. Document a versioned path map rather than scattering dozens of fragile path expressions across dashboards.

6.4.2 Casting nested values

Explicit casts serve two purposes: they provide a predictable SQL type and make an unapproved publisher change visible during transformation. Where unexpected input must be quarantined rather than halt every row, convert to a string and use the appropriate TRY_TO_* function, then record both the attempted input and the failure reason. A silent null is not a sufficient audit event.

Listing 7: Access a nested energy value and convert deliberately.

```
WITH src AS (
 SELECT PARSE_JSON(
   '{"reading":{"mwh":"12.500","unit":"MWh"}}') v
)
SELECT v:reading:unit::VARCHAR AS unit,
       TRY_TO_DECIMAL(v:reading:mwh::VARCHAR,18,3)
         AS energy_mwh
FROM src;
```

6.4.3 Null, missing, and unexpected type handling

A missing key produces SQL NULL during path extraction. A present JSON null can be distinguished with IS_NULL_VALUE; a value present with the wrong type must be handled as a separate contract failure. Consider how each state should appear in audit statistics, and avoid a single "failed to cast" counter that hides materially different root causes.

The decision at the dashboard boundary
For a measured energy total, SQL NULL, JSON null and a missing key may all yield no numeric value, but their cause is different. Preserve the cause in a quarantine reason code. For an optional comment field, a null result may be acceptable; for a required MWh measurement, it is not.

Check. If a supplier starts sending an `mwh` object instead of a numeric `mwh` value, identify the failing path and explain why applying COALESCE(...,0) before contract validation would conceal the error.

Project reference: P062: Nested multimodal extraction example. *Not independently executed; rights review pending.*

6.5 Unnesting arrays with FLATTEN and LATERAL

6.5.1 The FLATTEN table function

An array is often the natural API representation for a collection of measurements. An analytical fact table normally uses one row per observation. Snowflake's FLATTEN function bridges those shapes by emitting rows for the elements of a semi-structured container. Among the output columns are VALUE, INDEX, PATH and KEY. For array elements, INDEX records the original position: it is useful when auditing whether a publisher reordered or omitted a reading.

6.5.2 LATERAL usage and row cardinality

The LATERAL join permits FLATTEN to reference a field from the envelope row to its left. One envelope with three readings produces three flattened rows; two envelopes with three readings each produce six. An ordinary inner-style lateral flatten on an empty array produces no child rows. Consequently, counting resulting rows alone cannot prove that no input envelopes were lost. A separate envelope-level validation must reconcile accepted, empty and invalid arrays.

Listing 8: One output row per measurement in an envelope.

```
WITH src AS (
 SELECT PARSE_JSON(
  '{"observations":[{"tech":"wind","mwh":21.25},'
  || '{"tech":"solar","mwh":12.50}]}') AS doc
)
SELECT f.index AS source_array_index,
       f.value:tech::VARCHAR AS technology,
       f.value:mwh::NUMBER(18,3) AS mwh
FROM src s,
     LATERAL FLATTEN(INPUT => s.doc:observations) f;
```

6.5.3 Deep arrays and array position preservation

Chained LATERAL FLATTEN calls are appropriate for nested arrays only when the mapping needs both levels. For example, a response may contain regions, each with hourly readings. Record the outer region identifier and inner reading position before aggregating. Recursive flattening can explore irregular documents, but exploratory recursion is not a substitute for a specific, reviewed business grain.

Cardinality check.
If the contract promises three unique technologies per hourly envelope, require `COUNT(*) = 3` and `COUNT(DISTINCT technology) = 3` after flattening each envelope. A row count of six over two envelopes is necessary but not sufficient: a duplicate technology could replace a missing one.

Failure case. A publisher includes two arrays: readings and quality checks. Flattening both together may multiply rows. Flatten only the needed path; aggregate separate arrays at the correct grain.

Project reference: P063: Nested arrays from multimodal analysis. *Not independently executed; rights review pending.*

6.6 Handling schema drift and missing values

6.6.1 Compatible schema changes

Schema evolution is a contract negotiation, not an automatic casting trick. A new optional metadata field can be compatible with existing consumers. Renaming a required observations array may be compatible only if the mapper knows which provider schema versions use the old and new paths. Changing a unit or redefining the observation interval is potentially breaking even if the JSON structure remains identical. Version your publisher-to-canonical mapping and document the effective date.

Our synthetic energy feed illustrates this. Version 1 contains `observations`; version 2 contains `measurements`. Both map to the same canonical business grain. The mapper must inspect the declared `schema_version`; a naive COALESCE of two paths is convenient for exploration but not a complete contract strategy when both fields can appear or have different semantics.

6.6.2 Defaulting versus quarantining records

Defaulting is appropriate only when the missing value has a documented and low-risk interpretation. An absent optional quality comment may be defaulted to an empty display string. Missing `mwh`, a non-numeric reading, or an unrecognized unit must be quarantined. Setting missing measurements to zero suppresses errors and fabricates a measurement. Record at least the offending envelope, path, expected type, observed type and failed rule.

6.6.3 Tracking missing fields and attribute drift

Maintain an observable schema signature: version, required-key coverage, keys seen, type distribution, unknown paths and null frequency. Compute it over received envelopes *before* publishing silver rows. Differences between yesterday and today are signals for review, not always automatic failures. A whitelist of explicitly supported versions is often more reliable than guessing based on the presence of individual keys.

Change	Assessment	Outcome
New optional tag	Nonbreaking if documented	Preserve in bronze
`observations` renamed	Versioned mapper needed	Normalize after review
`mwh` becomes `null`	Required numeric absent	Quarantine reading
`unit` changes to kWh	Semantic contract change	Convert only if authorized

Production lesson. Successful query completion is not evidence of unchanged semantics. Monitor both the physical event stream and the accepted record stream. An increasing percentage of quarantined rows should raise a source-quality alert before the dashboard quietly loses coverage.

Project reference: P064: ShareWatch schema-drift reference. *Not independently executed; rights review pending.*

6.7 Parquet, mixed-format sources, and normalization

6.7.1 Columnar Parquet payloads

JSON is convenient for transporting variable nested records. Parquet is a column-oriented file format designed for efficient storage and analytics, with schema and encoding metadata. It commonly appears in cloud landing zones and lakehouse architectures. Parquet can itself contain nested structures; do not assume a Parquet file is already a clean relational table. Snowflake supports loading and transforming staged Parquet data through appropriate file-format and `COPY INTO` workflows.

6.7.2 Reconciling mixed schemas

Suppose two publishers provide conceptually identical electricity readings: API A sends JSON with a numeric `mwh`; export B sends Parquet with a field named `energy_kwh`. Both should map through source-specific adapters into one canonical model. Normalize units, timestamps, source IDs and technology vocabulary explicitly. Combining files first and hoping implicit casts fix their differences makes lineage hard to explain and mistakes hard to reverse.

Listing 9: Illustrative typed mapping after a reviewed conversion.

```
-- For an authorized source that declares kWh:
SELECT TRY_TO_DECIMAL(energy_kwh::VARCHAR,18,3)
         / 1000 AS energy_mwh
FROM approved_parquet_staging;
-- This query assumes a previously created typed staging
-- relation. It is not the statement that loads Parquet.
```

6.7.3 Typed silver-layer normalization

Define a transport-independent silver contract: provider, technology, UTC interval, energy in MWh, source envelope ID, array position, and schema version. Persist validated measurements with decimal precision suitable for the use case. For a production system, retain a documented rounding policy and reject non-finite or out-of-range values; do not trust automatic format conversion to determine financial or scientific precision.

Compatibility is a property of the contract, not the extension.
A well-defined JSON adapter and a well-defined Parquet adapter may produce identical silver rows. A badly documented Parquet file can be less trustworthy than a carefully documented JSON feed. Data quality is demonstrated by tests and provenance, not guaranteed by a file format.

Reader check. Explain how you would compare two publishers with different currency-like unit scales, duplicate correction policies and timestamp conventions. Which rules belong in the source adapters, and which belong in the canonical energy fact contract?

Project reference: P002: Iceberg table reference. *Not independently executed; rights review pending.*

Project reference: P065: Data crawler reference. *Not independently executed; rights review pending.*

6.8 Performance and modeling considerations

6.8.1 Selective projection and flattening cost

The most expensive semi-structured transformation is sometimes the unnecessary one. A single API document with hundreds of attributes may be stored and inspected without flattening every nested array. Select only paths required by the consumer and preserve the rest in bronze. Flattening an array with n elements produces n child rows; flattening two independent arrays with m and n elements in the same row can produce $m \times n$ combinations. Multiplication is a correctness problem before it becomes a performance problem.

For the energy application, suppose 100,000 hourly envelopes each hold ten technology readings. The theoretical full flattening produces one million rows before filters and later grouping. This is an illustrative cardinality calculation, not a measured Snowflake benchmark. If a report needs only wind power, project that technology deliberately and compare filtering in the nested representation with filtering a typed silver table. The best implementation depends on the data distribution, workload and storage layout.

Listing 10: Project only the contracted columns needed downstream.

```
SELECT e.envelope_id,
       f.index AS source_position,
       f.value:tech::VARCHAR AS technology,
       TRY_TO_DECIMAL(f.value:mwh::VARCHAR,18,3)
         AS energy_mwh
FROM accepted_envelopes e,
  LATERAL FLATTEN(INPUT => e.payload:observations) f
WHERE f.value:tech::VARCHAR = 'wind';
-- Illustrative query: accepted_envelopes is a lab view.
```

In an environment with frequent queries, repeated casting and flattening can add latency and cost. It can be cheaper operationally to materialize selected, validated fields into a stable silver table. But materialization introduces its own refresh, storage, reconciliation and deletion responsibilities; make that trade-off explicit rather than assuming duplication is free.

Measure the real workload.
Performance claims require comparable query text, warehouse settings, warm/cold cache disclosure, data volume, time window and correctness checks. Never describe a faster result as an optimization if it returns fewer valid readings. Chapter 12 develops a controlled measurement method.

Project reference: P066: Document extraction modelling reference. *Not independently executed; rights review pending.*

6.8.2 Pruning limits with nested attributes

Snowflake stores data using columnar optimizations, including extraction of some semi-structured paths. Nevertheless, not every nested attribute behaves like a dedicated relational column. In particular, inconsistent types, JSON null values and complex arrays can interfere with efficient extraction of individual paths. Snowflake's published considerations recommend promoting some frequently filtered or non-native fields into native columns after evaluating a representative workload.

The implication is practical: `payload:observed_at` is often a string in source JSON, whereas `observed_at_utc TIMESTAMP_TZ` is a typed column in silver. A query filtering one UTC day may perform differently across these layouts. It is not enough to infer from SQL readability that the storage engine will scan identical amounts of data.

Design	Consequence to evaluate
Raw `VARIANT` only	Maximum source flexibility; repeated path access and casting
Nested path projection	Avoids irrelevant fields; still depends on nested representation
Typed silver columns	Predictable types, joins and filters; additional refresh/storage
Flattened child table	Reusable one-row-per-reading grain; expansion and maintenance

The correct comparison measures the same answer over the same accepted business keys. If `VARIANT` contains seven records but silver quarantines one invalid record, raw and silver totals cannot be compared as equivalent workloads until the eligibility filter is aligned. Explain this invariant before teaching Query Profile: otherwise readers may accidentally reward the query that silently drops bad data.

The limits of intuition. Nested paths are not automatically slow; Snowflake documents cases where relational and well-behaved semi-structured data show similar performance. Nor is a flattened schema automatically superior for sparse, high-cardinality optional attributes. The storage and compute decision should be revisited if query patterns shift, source versions multiply, or analysts repeatedly use the same three nested fields.

Experiment specification
Compare (A) `VARIANT` path filtering and (B) a typed silver table with identical validation rules. Record row counts, checksum or totals, scanned data, elapsed time, warehouse size, cache state, and estimated cost. Run repetitions and report the distribution rather than picking the fastest run.

6.8.3 Documented modeling and performance decisions

A production design decision becomes more useful when recorded as a small architecture decision record (ADR). For our synthetic energy platform, write down the context: a publisher sends nested hourly documents, a reporting team needs reliable aggregates, optional metadata evolves, and invalid records must remain traceable. Then state the decision: retain raw text and parsed JSON for audit; apply explicit versioned adapters; create typed silver readings for repeatable joins and reporting.

ADR-006 — Normalize high-value energy readings
Context: Nested API envelopes change over time; dashboards require stable UTC hourly facts.
Decision: Store immutable source text and VARIANT; reject unknown required versions; flatten approved observation arrays; materialize typed readings.
Consequences: Stronger audit and consumer stability; additional storage, change-control and pipeline maintenance.
Revisit when: Schema change frequency, workload, costs or required freshness differ materially.

Monitoring matters as much as speed. Track parse failure rate, unsupported-version rate, arrays with zero or excess elements, missing business keys, invalid MWh count, accepted observation count and data freshness. For every metric, agree on an owner and a response. A spike in unknown source fields should create a schema-change review; an unexplained fall in the accepted observation count should stop publication rather than merely produce a smaller chart.

The upstream source contract also affects recovery. If a malformed message is later republished correctly under the same publisher message ID, the new revision should be preserved rather than overwriting the earlier bytes. If the provider does not expose stable IDs, define an author-owned fingerprint from source, retrieval metadata and content hash, and record its limitations. Such a fingerprint identifies a received message, not necessarily a unique real-world observation.

What we can and cannot conclude. The examples in this chapter are original teaching material with deterministic synthetic expectations. They do not establish the best performance profile for a particular Snowflake edition, region, warehouse size or production source. External GitHub repositories listed as references are candidates for further study, not proof that every dependency, license or execution path has been independently checked.

Chapter decision summary.
Preserve raw bytes to know what arrived; parse without hiding failures; approve schema-version mappings; flatten with explicit row grain; cast with validation; and publish stable silver facts only when acceptance evidence is complete.

Project reference: P067: Automated document-derived pipeline example. *Not independently executed; rights review pending.*

6.L Guided lab: energy API payloads to silver facts

6.L.1 Inputs, prerequisites, and cost guardrails

This lab continues the *synthetic* Finland Energy Analytics case from the earlier chapters. The data are not observations from Fingrid or another real publisher. Four JSONL physical lines represent: a valid v1 document with three technology measurements; a valid v2 document with a renamed observations array; a semantically invalid document whose energy value is JSON null; and a syntactically malformed document. The lab tests whether these four outcomes can be traced without turning invalid readings into zero.

Expected offline fixture: 4 physical lines; 2 accepted envelopes; 1 contract-quarantined envelope; 1 syntax-quarantined envelope; 6 accepted observations; accepted total **97.500 MWh**. Exact values are stored in the supplied `data/sample_api_payload.jsonl` and checked by `tests/test_payloads.py`. This is a deterministic learning fixture, not a published energy statistic.

Cloud execution is optional and billable. Use a disposable schema, a limited role and auto-suspending warehouse. Do not commit credentials. Save query IDs and results for review.

6.L.2 Step-by-step procedure and selected code

Step 1: Inspect each raw line and its line number. **Step 2:** Execute the offline Python validator; check accepted, quarantined, and normalized counts. **Step 3:** In an approved Snowflake lab, create `bronze_json_raw` and load the same raw lines as text, preserving the line index. **Step 4:** Use `TRY_PARSE_JSON`, supported versions and expected array path to classify envelopes. **Step 5:** Flatten accepted arrays and cast the values to a typed silver relation.

Listing 11: The core extraction; full script and guards in companion lab.

```
WITH parsed AS (
 SELECT source_line, TRY_PARSE_JSON(raw_line) AS payload
 FROM bronze_json_raw
), approved AS (
 SELECT source_line, payload,
        CASE payload:schema_version::NUMBER
          WHEN 1 THEN payload:observations
          WHEN 2 THEN payload:measurements
        END AS readings
 FROM parsed
 WHERE IS_OBJECT(payload)
)
SELECT source_line, f.index AS array_position,
       f.value:tech::VARCHAR AS technology,
       TRY_TO_DECIMAL(f.value:mwh::VARCHAR,18,3)
         AS energy_mwh
FROM approved a,
     LATERAL FLATTEN(INPUT => a.readings) f
WHERE IS_ARRAY(a.readings);
```

This extraction is *not* the full acceptance gate: the companion adds time, units, uniqueness and rejection tests. Preserve actual query IDs in a real account.

6.L.3 Expected results, failure diagnosis, and cleanup

The offline validator produces the following expected outcome from its four synthetic physical lines: **2 accepted envelopes**, **1 semantic rejection**, **1 syntax rejection**, **6 accepted technology readings**, and **97.500 MWh**. These expectations were calculated from the fixture and are cross-checked by the Python tests. They are not presented as output from an executed Snowflake warehouse.

If Snowflake counts disagree, investigate in this order: (1) raw-line count and content hash; (2) parser classification, including JSON versus SQL null; (3) approved schema versions; (4) selected array path; (5) number of elements and array positions; (6) numeric/unit/time contracts; (7) duplicate accepted business keys; (8) reconciliation to the published silver table. Do not replace a failed assertion with a blind `DISTINCT`, nor use `COALESCE(mwh, 0)` as an error correction.

Cleanup. Keep the original fixture and test report for reproduction. The companion SQL cleanup file contains only commented statements affecting chapter-specific objects; review ownership and object names before manually enabling anything destructive. Suspend lab compute after use. Optional query logging should preserve IDs and schema version, not credentials or personal data.

6.L.4 Exercises and assessment

1. **Foundation:** Explain why JSON null, a missing `mwh` key, and invalid JSON need separate error classifications. Provide one example of each.
2. **Practitioner:** Add a valid v3 envelope using `readings` instead of `measurements`. Implement an explicit v3 mapper and update the expected total and tests; preserve the earlier fixtures unchanged.
3. **Architecture:** Design a bronze-to-silver migration when a source changes from MWh numbers to `{"value":12500,"unit":"kWh"}` objects. Identify what is backward-compatible and what requires approval.

Scoring (10 points). Raw provenance (2); parser/shape classification (2); typed conversion and units (2); array cardinality and uniqueness (2); recovery and proof of verification (2). Passing requires at least 8 points and no hidden null-to-zero replacement.

> **The chapter in one principle.** JSON flexibility belongs at a controlled ingestion boundary. Analytical meaning must remain stable even when the source representation evolves. Chapter 7 extends that boundary from batch messages to streaming arrival and freshness guarantees.

Official documentation and practical references (checked October 2026).

[1] PARSE_JSON and TRY_PARSE_JSON; [2] VARIANT design considerations; [3] FLATTEN; [4] GET_PATH; [5] OBJECT_CONSTRUCT_KEEP_NULL; [6] Transforming data during load; [7] Parquet loading tutorial.